CadQuarry: A Permissively-Licensed, Procedurally-Generated Dataset of Parametric CAD Programs

Jacob Jennings

Independent Researcher | jacob.r.jennings@gmail.com

github.com/jacobjennings/CadQuarry | huggingface.co/datasets/jacobjennings/cadquarry

Code: Apache-2.0 | Data: cc0-1.0 (public domain) | June 10, 2026

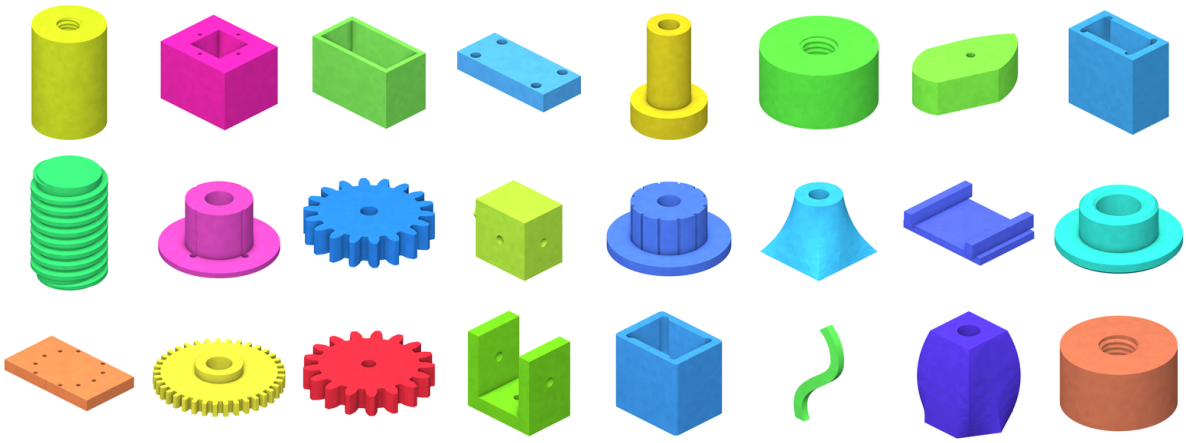


Figure 1. A 24-part sample drawn across CadQuarry's fifteen part families. Every object is a deterministic function of a single integer seed, executes to a watertight solid, and ships as live-editable parametric CadQuery code under a cc0-1.0 public-domain dedication. Images are CadQuarry's own headless GPU renders (Section 5).

Abstract. Progress on learning-based CAD generation is gated by data whose licensing is restrictive or ambiguous and whose programs are fixed instances rather than editable designs. We present **CadQuarry**, a procedural generator and the corpus it produces: diverse, executable, *parametric* CadQuery programs released as public-domain (cc0-1.0) data with an Apache-2.0 generator. Each part is a typed parameter schema plus a pure `build(p)` function, so any sample is a live design whose geometry updates in milliseconds, not a frozen mesh. The vocabulary spans plates, brackets, shafts, blocks, multi-section assemblies, freeform 2D sketches (line/arc/Bézier/spline loops, extruded or revolved), lofted reducers, swept profiles, and three solver-built mechanical families—involute/cycloid *gears*, standards-based *threads* (ISO/ACME/trapezoidal), and *tapped* holes with real cut internal threads—raising the complexity ceiling beyond the fixed sketch-extrude sequences of prior corpora. Validity is guaranteed by execution and duplicates are removed by a rotation/translation-invariant geometry signature; across our largest corpus all 200,000 accepted parts are geometrically distinct. The data is published on the Hugging Face Hub as a size ladder (1k–200k) in six content configs—code-only JSONL plus Parquet with binary STEP, STL, and eight-view GPU renders—and is browsable through a committed in-browser gallery, and any corpus can be regenerated from a versioned seed list. Generation sustains ~10 validated parts/s on a 24-core workstation, paced by the solver-built mechanical families.

1 Introduction

Data-driven CAD—generating, completing, or recovering parametric solid models with neural networks—has advanced quickly, but the field's datasets carry two practical frictions.

Licensing. Today's CAD datasets are, almost without exception, either non-commercial or bound by unsettled per-model terms. The large collected corpora—ABC [1] and the sketch-extrude sequences derived from it (DeepCAD [2] and its text-augmented descendant Text2CAD [3])—are parsed from Onshape public documents, so their geometry stays copyright its creators under those source terms; the human-authored Fusion 360 Gallery [4] ships under a custom Autodesk *non-commercial* research license, and even the procedurally generated CAD-Recode [5] is released CC BY-NC. Unrestricted—especially commercial—reuse is therefore a case-by-case legal question rather than a settled fact (Table 1).

Editability. Most released CAD datasets are *instances*: a fixed construction sequence or B-rep for one shape. The closest analog to this work, CAD-Recode [5], takes the elegant step of representing each model as CadQuery Python and procedurally generating a million of them—but those programs likewise encode single, fixed sketch-extrude solids. A program that is already a function of named, range-bounded parameters is strictly more useful: it is a design, not a snapshot.

CadQuarry targets both frictions directly, and pushes on the complexity ceiling while it does so.

Contributions

1. A **procedural generator** (Apache-2.0) that emits diverse, valid, *parametric* CadQuery programs, and the **corpus** it produces, released as public-domain cc0-1.0 data—clean for any use, commercial included.
2. **Parametric-by-construction** programs: a machine-readable `PARAMS` schema plus a pure `build(p)` function. Samples are live-editable designs, enabling research on customizable parts, data augmentation, and parameter inference.
3. A **raised complexity ceiling**: freeform sketched profiles, lofts and sweeps, multi-section assemblies, and three solver-built mechanical families—involute/cycloid gears, standards-based threads, and tapped holes—bridged into a single CadQuery solid.
4. **Validity by execution** and **geometry-signature dedup**; empirically, 200,000/200,000 accepted parts are distinct.
5. **Developer experience**: a Hugging Face size ladder in Parquet/JSONL configs, **eight-view** GPU renders, and a no-install in-browser gallery and live customizer.
6. **An extensible generator built for a community**: because every corpus is a function of an Apache-2.0 generator and a versioned seed list, a new family or an improved sampler is just a new generator version—inviting others to iterate on and grow the design space rather than re-scrape it, with no proprietary pipeline in the loop.

2 Related CAD Datasets

Table 1 situates CadQuarry among widely used CAD datasets. We follow the *Datasheets for Datasets* [6] framing: because our contribution is the data and tooling rather than a model, the rest of the paper is organized as a datasheet—generation methodology, validity characterization, distribution statistics, intended uses, and licensing—rather than as a methods paper. CAD-Recode [5] and Fusion 360 Gallery [4] are the closest references for framing generation and tasks; recent work such as CADmium [7] illustrates the geometric-quality evaluation our data is meant to support.

The differentiators we foreground are not claimed elsewhere as primary contributions: an unambiguous public-domain grant, live parametric editability, and real mechanical geometry (threads, gears, assemblies) alongside the usual prismatic and revolved parts.

3 Generation Pipeline

CadQuarry samples an intermediate representation (IR), emits self-contained CadQuery source, executes it in a sandbox, and accepts the part only if it builds to a single valid solid that is not a geometric duplicate of one already seen (Fig. 2). The IR, emitter, composer, and deduplicator have no dependency on CadQuery itself; only execution does.

Dataset	Scale	Representation	Origin	Param.[a]	Data license[b]
ABC [1]	~1M	B-rep (STEP)	collected	✗	Onshape ToU[†]
DeepCAD [2]	~178k	sketch-extrude seq.	from ABC	✗	Onshape ToU[†]
Fusion 360 [4]	8,625	seq. + timeline	human	✗	non-comm. res.
Text2CAD [3]	~170k	seq. + text	aug. DeepCAD	✗	CC BY-NC-SA
CAD-Recode [5]	1M	CadQuery code (fixed)	procedural	✗	CC BY-NC
CadQuarry	up to 200k[c]	**parametric CadQuery** + B-rep, STL, renders	procedural	✓	**cc0-1.0**

Table 1. CadQuarry in context. [a] Programs are functions of named, range-bounded parameters (live-editable), not fixed instances. [b] Data license *as published* (June 2026); verify at source. [†] ABC and DeepCAD geometry is parsed from Onshape public documents and remains copyright its creators under Onshape's Terms of Use—the repositories' MIT license covers their code, not the models. Fusion 360 Gallery is non-commercial research-only (no full redistribution); Text2CAD is CC BY-NC-SA 4.0; CAD-Recode's corpus is CC BY-NC 4.0. CadQuarry is the only public-domain (cc0-1.0) corpus here—the only one usable commercially with no attribution. [c] Published ladder; the generator is unbounded, and any corpus is reproducible from its seed.

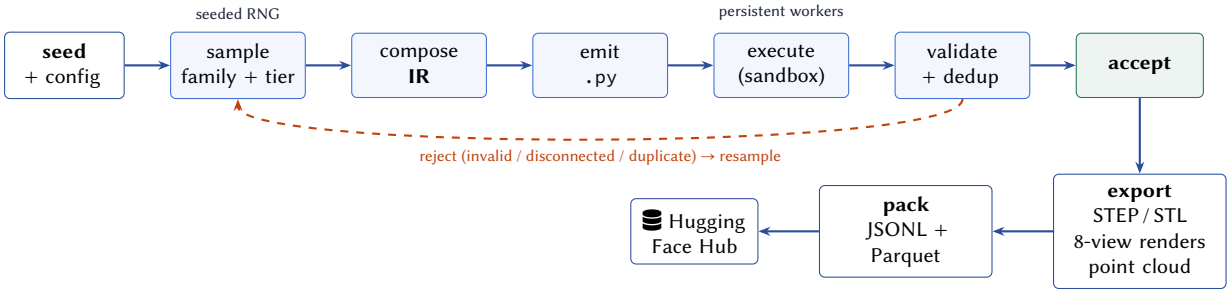


Figure 2. The generation pipeline. Sampling is seeded per part and the accept/reject decision is taken in strict attempt order, so the corpus is identical regardless of how many workers run. Text artifacts (code, params, meta) are produced by `generate`; geometry and renders are produced on demand by `export`.

3.1 Part families and complexity tiers

Fifteen families span a deliberately broad operation vocabulary (Fig. 3). Twelve are self-contained on CadQuery alone; the three mechanical families are described in Section 4.1. Each part also draws a *complexity tier* (0–3) that governs how many features it carries—from a bare primitive (tier 0) to deep multi-section trees with pockets, bosses, ribs, and extra ports (tier 3). Tiers are clamped per family (e.g. brackets and assemblies start at tier 1), and the compound family scales its section count with tier, so a manifold sprouts additional bored ports as complexity rises.

3.2 Parametric program format

Every emitted `.py` is self-contained and customizer-compatible: a typed, range-bounded, UI-labeled `PARAMS` schema and a pure `build(p)` that is a function of those parameters (Fig. 4). This is the property that turns a sample from a frozen instance into a *design*—the same program yields a continuum of valid parts, which is what makes the data useful for customizable-part research, parameter inference, and on-the-fly augmentation. To be precise about scope: each published record is one program materialized at its *default* parameters—the geometry, eight-view renders, and dimension summary all describe the part `build(p)` produces under those defaults. We deliberately do *not* enumerate the combinatorial space of parameter settings into the corpus; doing so would bloat it with near-duplicates and defeat dedup. The point of shipping the `PARAMS` schema and the `build(p)` function is to invite models to *generate* parametric programs, not to pre-render every output of one. A sidecar `.params.json`

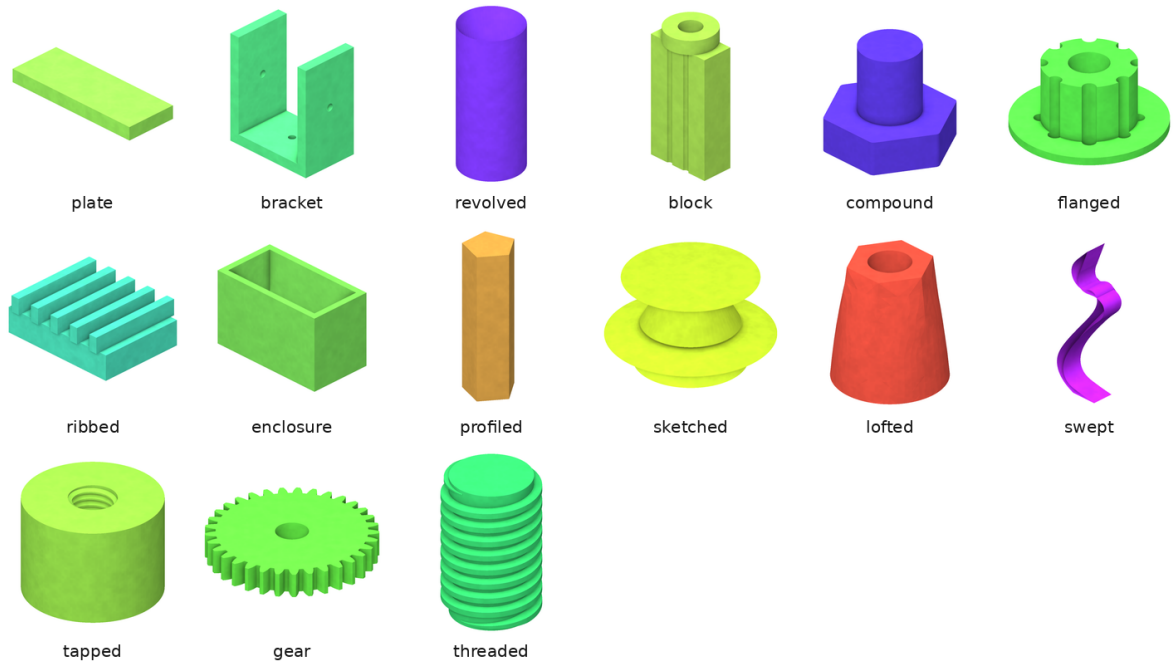


Figure 3. One representative per family (default parameters, `iso_fr` render). plate, bracket (L/C/Z angle brackets with gussets), revolved shafts/bushings, block housings, compound multi-section assemblies, flanged bolt-circle hubs, ribbed structures, shelled enclosures, profiled bars, sketched freeform line/arc/Bézier/spline bodies, lofted reducers, swept profiles, tapped bodies, involute gears, and standards-based threaded parts.

mirrors the schema for consumers that do not execute Python, and—for families with categorical design intent—a machine-readable `descriptor` resolves the part's defining attributes (e.g. a thread's `M16x2` designation, a gear's module and tooth count) so they can be queried without parsing source. Every part additionally carries a procedurally-generated, plain-English *dimension summary*—a short, prompt-friendly description of its overall size and each feature (holes, fillets, bores, pockets, brackets, gears, threads), resolved against the default parameters. Crucially this is *not* AI-written text: it is a pure, deterministic walk of the IR (e.g. *"Rectangular body, 55× 23.5× 75.4 mm (W×D×H). Four Ø3.2 mm holes at the corners, 38.5× 16.45 mm spacing. Fillet radius 1.078 mm on vertical edges."*), so it is deterministic and pairs with the renders (Section 5) as a faithful text–image signal for captioning and program-recovery work. The summary is emitted as a `dimensions` field in the metadata, a `dims/` text sidecar, and a column of the published tables.

4 Validity, Deduplication, and Reproducibility

Validity by execution. CadQuarry never ships a part it has not built. A candidate is accepted only if its program runs to completion and yields exactly *one* connected solid of non-trivial volume, finite bounding box, and a plausible aspect ratio; multiple solids signal a union that failed to fuse and are rejected. This trades a little yield for the guarantee that every published program executes.

Geometry-signature dedup. To avoid near-trivial repeats, each solid is reduced to a signature of quantized, analytically computed invariants:

$$\sigma \;=\; \big(\,V,\;A,\;\mathrm{sort}(I_1, I_2, I_3),\;n_f,\;n_e,\;n_v\,\big),$$

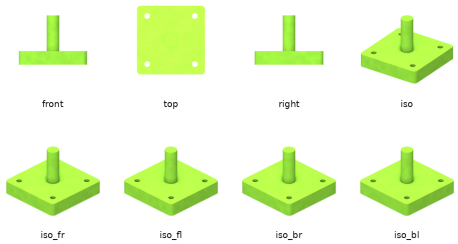
volume V, surface area A, the sorted principal moments of inertia I_k, and face/edge/vertex counts, each rounded to three significant figures. The principal moments—eigenvalues of the inertia tensor about the centroid—are invariant to rotation and translation and are sorted before quantization, so orientation and placement never matter. Quantities are computed from the B-rep (not mesh-sampled), so the signature

```python
import cadquery as cq
# CadQuarry v0.6.0 - seed 1234003889 - CC0-1.0

PARAMS = {
  "base_len":  {"type": "float", "default": 60.0,
     "min": 25, "max": 90, "group": "Body",
     "label": "Base leg length (mm)"},
  "n_holes":   {"type": "int", "default": 2,
     "min": 1, "max": 3, "group": "Holes",
     "label": "Holes per leg"},
  "gusset":    {"type": "bool", "default": True,
     "group": "Gusset", "label": "Corner gusset"},
  # ... 5 more typed, range-bounded params
}

def build(p):
    base  = cq.Workplane("XY").box(
        p["base_len"], p["bracket_w"], p["bracket_t"],
        centered=(False, True, False))
    vert1 = cq.Workplane("XY").box(...)    # near leg
    vert2 = cq.Workplane("XY").box(...)    # far leg
    result = base.union(vert1).union(vert2)
    if p["gusset"]:
        result = result.union(...)         # corner
        ↪ gussets
    return result   # -> one watertight solid

result = build({k: v["default"]
                for k, v in PARAMS.items()})
```



The eight standard viewpoints of the part this program builds at its default parameters: three orthographic (`front`/`top`/`right`), a canonical `iso`, and the four isometric corners.

Figure 4. An abbreviated but faithful generated bracket program (left) and the eight-view render of the solid it produces (right). Moving any slider re-runs `build` and updates all geometry; nothing is pre-baked.

is deterministic. A part whose σ has been seen is dropped.

Empirical diversity

On the published **200k** corpus, **all 200,000** accepted parts have distinct geometry signatures—0 collisions at the dedup tolerance. Dedup also acts as a natural governor on families with a small design space: the threaded family, sampled at ~5%, realizes a smaller share of the corpus, because the finite ISO/ACME standards space saturates and later collisions are dropped.

Reproducibility. A versioned seed list pins each published corpus to a tuple of `seed`, `count`, `config`, and generator version, so the published Parquet/JSONL is a *convenience artifact*: the generator plus the seed list is the canonical source, and any corpus can be regenerated rather than relying on a hosted copy. The geometry signature is itself versioned, so the dedup rule travels with the seed list.

4.1 Mechanical families: gears, threads, and tapped holes

The gear, threaded, and tapped families build real, standards-based geometry through the build123d [8] stack and bridge it back into CadQuery via each object's shared OCP `.wrapped` handle—so after one line a generated gear or thread is an ordinary single-solid `cq.Workplane` that flows through the same execution, dedup, export, and customizer paths as every other family. Gears (via py_gearworks [9]) cover spur, helical (including herringbone), bevel, cycloid, and inside-ring types on the ISO 54 module series, with optional profile shift, root fillets, bore, and hub. Threads (via bd_warehouse [10]) cover ISO metric external and internal threads from the ISO 261/262 pitch tables, plus ACME and trapezoidal lead screws by standard designation. The tapped family reuses bd_warehouse to cut real ISO internal threads into a plate, block, or cylinder: for each hole the body is bored to the thread root and an internal

thread is fused in before the bridge, yielding one valid solid with genuine thread geometry. Their seeded *selection* and dedup are stable like any other family; only the exact solver output is platform-dependent (Section 10).

5 Eight-View Rendering

Each part is rendered from eight fixed viewpoints—`front`, `top`, `right`, a canonical `iso`, and the four isometric corners (Fig. 4, right)—to give learners and humans a stable multi-view signal. Rendering runs on the GPU through a headless EGL OpenGL context (`moderngl`) using a small deferred pipeline rather than a painter's-algorithm mesh plot:

(1) a **geometry pass** writes view-space position and a flat geometric normal (from screen-space derivatives) into a G-buffer with a true depth buffer, so no back/internal faces bleed through;
(2) an **SSAO pass** (24-sample hemisphere kernel) adds contact darkening in crevices and along edges—the dominant "CAD" depth cue—then blurs it;
(3) a **lighting pass** composites soft hemispherical ambient plus a single wrapped positional key light, so shading varies gently across flat faces instead of reading dead-flat, with a faint procedural matte grain; and
(4) everything renders at 2× supersampling and is box-downsampled on the GPU with premultiplied alpha over a transparent background for clean anti-aliased edges.

The EGL context and all framebuffers are created once per process and reused across parts and views, so the per-part cost is a vertex upload and a few draw calls; a per-part color is hashed deterministically from its id so renders are stable regardless of order or parallelism. Rendering parallelizes across processes that share one GPU.

6 The Corpus: Records, Distribution, and Formats

6.1 What each record contains

Every part is one record carrying four kinds of content: the parametric *program*, a *natural-language* description, optional *geometry*, and *provenance* metadata (Table 2). The program and metadata are always present; geometry (STEP/STL/renders) is included only in the geometry configs (Table 3).

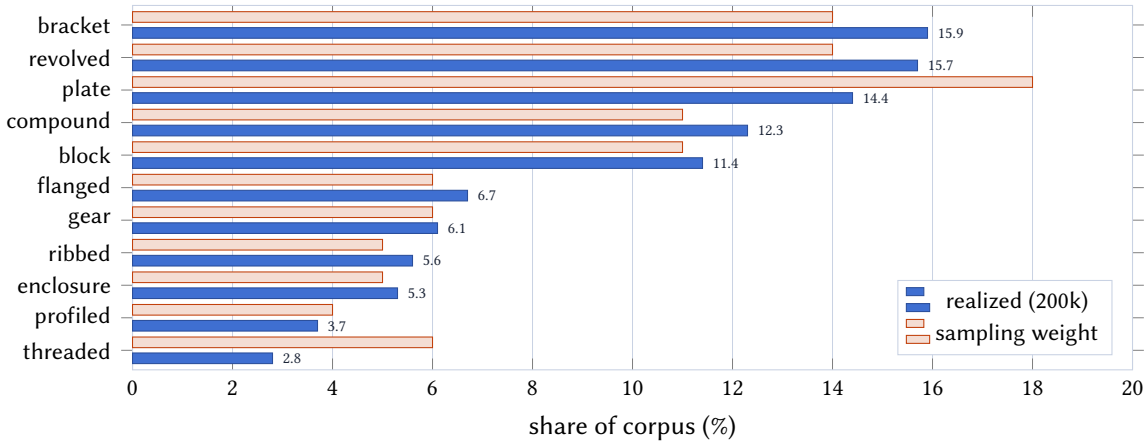


Figure 5. Realized family distribution on the 200k corpus vs. the configured sampling weights. Realized shares drift below weight for low-diversity families (threaded, profiled) because dedup removes saturating repeats, and above weight where tier clamps add parts (bracket). All weights are tunable in `configs/default.toml`.

6.2 Distribution

Figure 5 shows the realized family mix; complexity tiers on the same corpus land at **11.0%/49.6%/30.3%/9.0%** for tiers 0–3, reflecting the per-family tier clamps that pull the bare-primitive share down from its nominal 25%. These are characterization numbers, not targets; every knob is exposed in config.

Field	Contents
Program & parameters	
`source`	The parametric CadQuery program—a typed `PARAMS` schema plus a pure `build(p)`.
`params`	The `PARAMS` schema itself: for each parameter a type, default, `min`/`max`/`step`, and UI group and label. Mirrored as a `.params.json` sidecar.
`descriptor`	Machine-readable categorical design intent, for the mechanical families: e.g. a gear's `kind`/`module`/`teeth`/`pitch_diameter`, a thread's `M10x1.5` designation, pitch, and major diameter.
Natural-language description	
`dimensions`	A deterministic, plain-English summary of overall size and each feature, as a `text` blob plus per-feature `lines` (Fig. 4). Also a `dims/` sidecar.
Geometry (geometry configs only)	
`step_bytes`	STEP B-rep solid.
`stl_bytes`	Triangulated STL mesh.
`render_*`	The eight fixed-viewpoint GPU renders (Section 5).
Provenance & structure	
`part_id`, `family`, `tier`	Stable identifier, part family, and complexity tier.
`seed`, `generator_version`, `license`	The integer seed, generator version, and CC0-1.0 grant that fix the part's origin.
`symmetry`, `op_count`, `ir_hash`	IR-level structure: symmetry class, operation count, and a hash of the intermediate representation.
`geometry_signature`	The dedup key (Section 4): volume, surface area, sorted principal moments, and face/edge/vertex counts.

Table 2. The fields of a single CadQuarry record. Program, description, and provenance ship in every config; geometry fields appear only in the geometry configs of Table 3.

6.3 Formats and the size ladder

The corpus is published on the Hugging Face Hub [11] as a size ladder—`1k`, `2k`, `5k`, `10k`, `20k`, `50k`, `100k`, `200k`—each in six content configs (Table 3) so a consumer fetches only what they need, from a few-megabyte code-only pull to full geometry. Code-only data is JSONL with the parametric `source` and `params` inlined; geometry configs are Snappy-compressed Parquet with typed binary columns (`render_*` images, `stl_bytes`, `step_bytes`). Each config is also published restricted to tiers 0–2 (a `-t0-2` suffix) for consumers who want to exclude the most complex parts.

7 Performance

Generation validates every part by execution. Importing the CadQuery/OCP kernel (~1–1.5 s) is a fixed startup cost that CadQuarry amortizes with a pool of *persistent* workers that import once and then build many parts each, fanning work across cores while keeping the accept/dedup decision in strict attempt order—so output is independent of worker count. Under the full default configuration, end-to-end throughput is paced by the three solver-built mechanical families (gear, threaded, and tapped, ~14% of the mix), which solve real involute/thread geometry at a meaningful fraction of a second per part; the primitive families alone build in milliseconds. Table 4 reports wall-clock times for execution-validated generation (no geometry export); throughput settles at ~8–10 validated parts/s and climbs slightly with corpus size as the worker warm-up amortizes.

Config	Contents	Format
`<tag>`	CadQuery `source` + metadata + `params`	JSONL
`<tag>-renders`	+ eight-view render images	Parquet (image)
`<tag>-stl`	+ binary STL mesh	Parquet (binary)
`<tag>-step`	+ binary STEP B-rep	Parquet (binary)
`<tag>-geo`	+ renders + STL	Parquet
`<tag>-full`	+ renders + STL + STEP	Parquet

Table 3. Six content configs per size `<tag>` (plus a tier-limited `-t0-2` variant of each). `load_dataset("jacobjennings/cadquarry", "50k-full")` pulls everything for the 50k corpus; `"50k"` alone pulls just code and metadata.

Parts	Time	Throughput
1,000	2 m 10 s	7.5 parts/s
2,000	3 m 30 s	9.5 parts/s
5,000	9 m 30 s	8.7 parts/s
10,000	18 m 45 s	8.9 parts/s
20,000	36 m 30 s	9.1 parts/s
50,000	1 h 30 m	9.2 parts/s
100,000	2 h 50 m	9.8 parts/s

Table 4. Generation time on an AMD Ryzen Threadripper 9960X (24C/48T) with the default 24-worker pool, full default config (mech families included), execution-validated and geometry export excluded. The optimal worker count is system-dependent and memory-bound at the largest sizes, so on machines with less RAM fewer workers than cores may be needed to avoid swapping.

8 Developer Experience and Community

CadQuarry is meant to be inspected and extended, not just downloaded.

No-install preview. A 1,000-part sample is committed to the repository and served as a self-contained in-browser gallery (Three.js, lazy-loaded geometry) via GitHub Pages, so anyone can inspect the format and the real geometry without installing anything.

Live customizer. `cadquarry serve` opens any part in a browser customizer: move the sliders defined by `PARAMS` and the 3D view updates in milliseconds—no regeneration, no ML—directly demonstrating the parametric contract. The same tool browses a generated corpus as a filterable gallery.

One-command pipeline. `build` generates and exports every size in the ladder; `publish` packs and uploads them as Hugging Face configs; `build_sample` refreshes the committed preview. Builds are resumable and secrets (the HF token) are read from the environment and never logged or committed.

Versioned generator for community improvement. Because the corpus is a function of the generator and a seed list, contributions compose cleanly: a new family or a fixed sampler becomes a new generator version with its own seed list entry, and anyone can reproduce or extend a corpus without a hosted service in the loop. The generator is Apache-2.0; there is no proprietary pipeline to reverse.

9 Generative-AI Disclosure

The dataset is not AI-generated. Every part is produced by deterministic procedural code and an exact geometry kernel; no language or diffusion model samples, edits, or selects any geometry. This is a defining difference from LLM-/diffusion-generated CAD: CadQuarry's outputs are reproducible from a seed and validated by execution, not sampled from a learned distribution.

The tooling was built with AI assistance. The generator and supporting code were developed by the author in collaboration with AI coding assistants, under the author's direction and review; this is recorded in the project's commit history. All behavior is covered by an automated test suite and, for geometry, by the execution-validity guarantee described in Section 4.

This manuscript was drafted with AI assistance and edited and verified by the author, who is responsible for its content.

10 Intended Uses, Limitations, and Licensing

Intended uses. Pre-training and evaluation for CAD code generation and program synthesis; point-cloud/mesh/multi-view → program recovery (à la CAD-Recode [5]); text-conditioned generation and captioning using the per-part plain-English dimension summaries (à la Text2CAD [3], but with deterministic, reproducible descriptions); parameter inference and customizable-part research that exploits the `build(p)` interface; and geometry-quality studies using the structural metrics that recent work emphasizes [7]. The multi-format ladder supports both lightweight code-only and full-geometry workflows.

Limitations. The vocabulary, while broad, is mechanical-part oriented and does not cover free-form/organic surfaces; tapered pipe threads (NPT/BSPT) are a documented future addition, not a present capability; solver-built families are byte-reproducible only against pinned `mech` versions (Section 4.1); and the distribution reflects our sampling choices, which consumers should re-weight as needed via config. The procedural design space is ours, so the corpus is not a substitute for the long tail of real-world, human-authored designs.

Licensing. Generator source is Apache-2.0; all generated data—programs, STEP, STL, renders, parameter schemas, and metadata—is dedicated to the public domain under CC0-1.0. The split is intentional and unambiguous: do anything with the data, commercial use included, with no attribution required. The two homes (an Apache-2.0 GitHub generator, a CC0-1.0 Hugging Face corpus) map the code/data license boundary onto the platforms.

11 Conclusion

CadQuarry is a small contribution with a clear shape: a public-domain corpus of *parametric*, executable CAD programs, a raised complexity ceiling that includes freeform sketches, lofts, sweeps, and real gears, threads, and tapped holes, an unambiguous license, and a reproducible, fast, inspectable generator. We hope the parametric-by-construction framing and the CC0-1.0 dedication make it a comfortable default for CAD-program learning, and that the versioned generator invites the community to extend the design space rather than re-scrape it.

References

[1] S. Koch et al. "ABC: A Big CAD Model Dataset for Geometric Deep Learning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019. URL: https://arxiv.org/abs/1812.06216.

[2] R. Wu, C. Xiao, and C. Zheng. "DeepCAD: A Deep Generative Network for Computer-Aided Design Models". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021. URL: https://arxiv.org/abs/2105.09492.

[3] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal. "Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/0e5b96f97c1813bb75f6c28532c2ecc7-Abstract-Conference.html.

[4] K. D. Willis, Y. Pu, J. Luo, H. Chu, T. Du, J. G. Lambourne, A. Solar-Lezama, and W. Matusik. "Fusion 360 Gallery: A Dataset and Environment for Programmatic CAD Construction from Human Design Sequences". In: *ACM Transactions on Graphics (TOG)* 40.4 (2021). URL: https://arxiv.org/abs/2010.02392.

[5] D. Rukhovich, E. Dupont, D. Mallis, K. Cherenkova, A. Kacem, and D. Aouada. "CAD-Recode: Reverse Engineering CAD Code from Point Clouds". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2025. URL: https://arxiv.org/abs/2412.14042.

[6] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford. "Datasheets for Datasets". In: *Communications of the ACM* 64.12 (2021), pp. 86–92.

[7] P. Govindarajan, D. Baldelli, J. Pathak, Q. Fournier, and S. Chandar. "CADmium: Fine-Tuning Code Language Models for Text-Driven Sequential CAD Design". In: *arXiv preprint arXiv:2507.09792* (2025). URL: https://arxiv.org/abs/2507.09792.

[8] R. Maitland and build123d Contributors. *build123d: A Python CAD Programming Library Built on the OpenCascade Geometry Kernel*. https://github.com/gumyr/build123d. 2024.

[9] py_gearworks Contributors. *py_gearworks: Parametric Involute and Cycloid Gear Generation*. https://github.com/GarryBGoode/py_gearworks. 2024.

[10] bd_warehouse Contributors. *bd_warehouse: Standards-Based Threads, Fasteners and Parts for build123d*. https://github.com/gumyr/bd_warehouse. 2024.

[11] Hugging Face. *Datasets: A Community Library for Natural Language Processing and Beyond*. https://github.com/huggingface/datasets. 2024.